



POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING

Master Degree in Automation Engineering

Mobile Robotics

REPORT

**Autonomous Multi-Robot Mapping via
Stochastic and Deterministic Exploration**

AUTHORS:

Cialdella Matteo

Febbraro Samuele

Matera Salvatore

Somma Daniele Pio

Contents

Introduction	1
1 Overview	2
2 Project Structure	4
3 Multi-robot Management	8
4 Exploration Logics	11
4.1 Map Merger	11
4.2 Stochastic Exploration	12
4.3 Deterministic Exploration	13
5 Simulations	14
Conclusions	17

Introduction

In modern robotics, autonomous mapping and exploration of unknown environments are fundamental challenges, particularly where human intervention is hazardous. While single-robot approaches are simple, they suffer from time inefficiency in large areas and lack fault tolerance. To overcome these limits, multi-robot systems parallelize the process through cooperative swarms. However, managing a swarm introduces complex challenges in coordinate synchronization and navigation conflict resolution. This project develops and analyzes a scalable simulation framework for autonomous multi-robot mapping and navigation. The system coordinates three TurtleBot3 Waffle robots, configured under isolated namespaces, within a custom indoor environment. Each agent gathers local occupancy grid data using the SLAM Toolbox and navigates via the Nav2 stack. A centralized map merger node continuously aggregates these local maps into a unified global map.

A central focus of this work is the comparative performance analysis of two distinct swarm coordination paradigms.

The first approach consists of a stochastic frontier-based strategy that introduces the assignment of random points to the nearest robot in terms of Euclidean distance. The second, on the other hand, is a deterministic approach based on candidate goal extraction and assignment using the A* algorithm.

In order to guarantee the total reproducibility of the test environment, the entire application stack was fully containerized using Docker, leveraging a custom image based on the ROS 2 Jazzy distribution.

Chapter 1

Overview

For this project, three Turtlebot3 wheeled robots were utilized, each equipped with a 2D LiDAR sensor used for environment localization and mapping. To control the behavior of the robots, ROS2 Jazzy was employed via a custom Docker image. The entire project was set up for simulation on Gazebo Harmonic; therefore, a Git repository of `turtlebot3_gazebo` was initially cloned. Although this repository includes several default simulation environments, a custom environment modeled using Autodesk Fusion was used instead. This environment consists of an enclosed room featuring corridors and obstacles of various sizes and shapes randomly positioned on the floor.

From the `turtlebot3_gazebo` package, the `.urdf` file of the Turtlebot3 Waffle was extracted to obtain a digital model of the robot with its relative inertial and collision properties. The `.sdf` file, on the other hand, provides the same description as the `.urdf` file but is specifically readable by the simulator, extended with the necessary plugins for simulating the robot's sensors. Finally, a bridge configuration file was used to enable communication between ROS2 and Gazebo, defining the topic names, data types, and the direction of the bridge.

To build a local map for each robot, a standard node implemented in the `slam_toolbox` package was used, specifically `online_async_launch.py`. To link the frames of the

individual maps to a parent node, a node from the `tf2_ros` package was utilized, namely the `static_transform_publisher`.

To implement autonomous robot navigation, a custom instance of Nav2 was deployed, which, in turn, utilizes standard nodes from the `navigation2` package.

Lastly, two nodes were implemented for environment mapping based on two distinct criteria: a stochastic logic and a deterministic logic.

Chapter 2

Project Structure

The key files and nodes for the project are shown and described below:

- **docker_ws/Dockerfile.prj**: File that define the Docker container image with all the necessary installation.
- **docker_ws/build.sh**: Script to build the Docker image.
- **ros_ws/src/multirobot_mapping/config/nav2_params.yaml**: Nav2 parameters configuration file, shared by all three robots.
- **ros_ws/src/multirobot_mapping/config/slam_toolbox_robotX.yaml**: SLAM parameters configuration file for the individual robotX.
- **ros_ws/src/multirobot_mapping/launch/deterministic_swarm.launch.py**: Launch file that start the deterministic exploration and mapping of the environment. It calls the launch file `spawn_tb3.launch.py` and the nodes `deterministic_explorer.py`, `map_merger.py`.
- **ros_ws/src/multirobot_mapping/launch/map.launch.py**: Launch file that start RViz2 to view the explored maps.

- **ros_ws/src/multirobot_mapping/launch/**
nav2_navigation.launch.py: Custom launch file for the navigation stack of Nav2.
- **ros_ws/src/multirobot_mapping/launch/publisher_tb3.launch.py**:
Launch file that start the node `robot_state_publisher` for the TurtleBot3.
- **ros_ws/src/multirobot_mapping/launch/spawn_tb3.launch.py**:
Launch file that spawns a TurtleBot3 robot in the Gazebo simulation environment, setting up the necessary ROS 2 nodes for SLAM and navigation (Nav2) and establishing the communication bridge between ROS 2 and Gazebo.
- **ros_ws/src/multirobot_mapping/launch/**
stochastic_swarm.launch.py: Launch file that start the stochastic exploration and mapping of the environment. It calls the launch file `spawn_tb3.launch.py` and the nodes `stochastic_explorer.py`, `map_merger.py`.
- **ros_ws/src/multirobot_mapping/launch/**
utils.py: Script to loads a base Gazebo-ROS bridge YAML file and rewrites the topic names to include a robot-specific namespace, updates message types from `TwistStamped` to `Twist` and loads an SDF model file and to update its topics and frames definitions by prefixing them with a namespace.
- **ros_ws/src/multirobot_mapping/multirobot_mapping/**
deterministic_explorer.py: ROS 2 node that coordinates the deterministic exploration of the robots. The node subscribes to the merged global map, identifies unexplored frontiers, and assigns them to idle robots. It also monitors the robots' progress.
- **ros_ws/src/multirobot_mapping/multirobot_mapping/**
map_merger.py: ROS 2 node that merges the local maps of three robots

into a single global map.

- **ros_ws/src/multirobot_mapping/multirobot_mapping/stochastic_explorer.py**: ROS 2 node that coordinates the stochastic exploration of the robots. The node subscribes to the merged global map, identifies unexplored frontiers, and assigns them to idle robots. It also monitors the robots' progress.
- **ros_ws/src/multirobot_mapping/rviz/map_view.rviz**: RViz2 configuration script designed to visualize the maps generated by two types of exploration.
- **ros_ws/src/multirobot_mapping/rviz/three_robots_view.rviz**: RViz2 configuration script designed to visualize the three robots during exploration.
- **ros_ws/src/turtbot3_gazebo/models/stanza/stanza.sdf**: File to import and define the custom environment within the simulator, based on the 3D model `stanza.obj`.
- **ros_ws/src/turtbot3_gazebo/worlds/stanza.world**: File that represents the global simulation environment within the simulator. It configures the laws of physics, loads Gazebo's software engines and set the custom room position.

The project directory tree is shown below:

```
Multirobot_mapping/
├── docker_ws/
│   ├── build.sh
│   ├── build_mac.sh
│   ├── entrypoint.sh
│   └── Dockerfile.prj
├── ros_ws/
│   └── src/
│       ├── multirobot_mapping/
│       │   ├── config/
│       │   │   ├── nav2_params.yaml
│       │   │   ├── slam_toolbox_robot1.yaml
│       │   │   ├── slam_toolbox_robot2.yaml
│       │   │   └── slam_toolbox_robot3.yaml
│       │   ├── launch/
│       │   │   ├── deterministic_swarm.launch.py
│       │   │   ├── map.launch.py
│       │   │   ├── nav2_navigation.launch.py
│       │   │   ├── publisher_tb3.launch.py
│       │   │   ├── spawn_tb3.launch.py
│       │   │   ├── stochastic_swarm.launch.py
│       │   │   └── utils.py
│       │   ├── multirobot_mapping/
│       │   │   ├── deterministic_explorer.py
│       │   │   ├── map_merger.py
│       │   │   └── stochastic_explorer.py
│       │   ├── rviz/
│       │   │   ├── map_view.rviz
│       │   │   └── three_robots_view.rviz
│       │   └── turtlebot3_gazebo/
│       │       ├── models/
│       │       │   ├── stanza/
│       │       │   │   ├── meshes/
│       │       │   │   │   └── stanza.obj
│       │       │   │   └── stanza.sdf
│       │       │   └── worlds/
│       │       │       └── stanza.world
│       │       └── ...
│       ├── run.sh
│       ├── runmac.sh
│       ├── chown_me.sh
│       └── exec.sh
```

Chapter 3

Multi-robot Management

The hardest challenge of the project was the multi-robot management, which co-operated to completely map the environment having zero prior knowledge.

Multi-agent management in ROS2 can be achieved through the use of unique namespaces, which are strings used as prefixes for the robot topics and frames to which they refer.

In this project, namespace management was achieved through the launch file `spawn_tb3.launch.py`, by acquiring a "namespace" launch parameter and converting it into a string, it is able to start all the necessary nodes, implementing the multi-robot logic.

The first operation performed in this file refers to a modified script from `arshadlab/tb3_multi_robot` that, through two functions, allows rewriting the SDF and YAML file. The first function used was `load_sdf_with_namespace`, which loads an SDF model file and updates its topic definitions by prefixing them with a namespace. The modification also updates some frames, such as `odom`, `base_footprint` and `base_scan`, to ensure the correct connection of the TF tree.

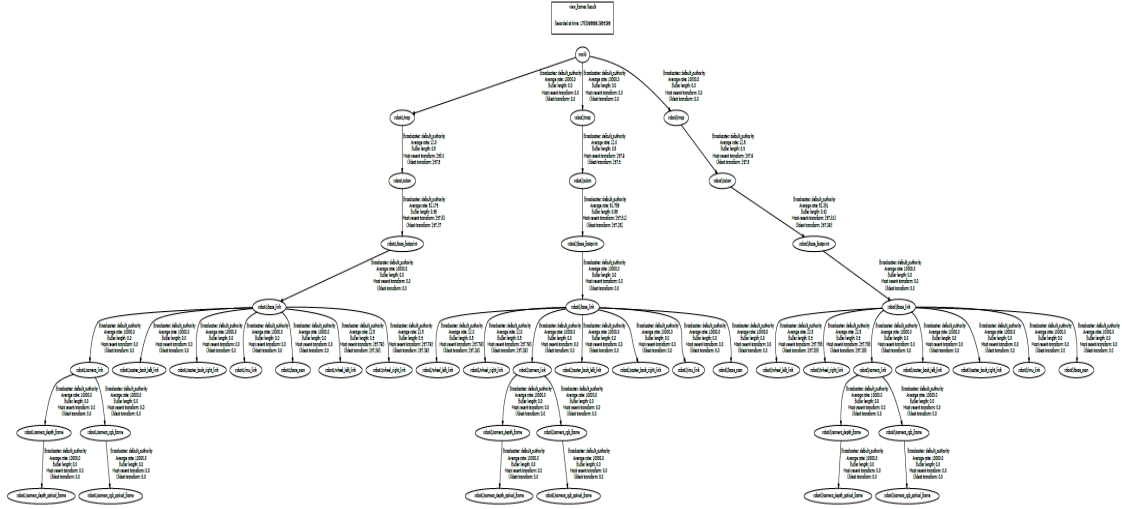


Figure 3.1: TF tree

The TF tree provides information regarding the hierarchy governing the robot frames, thereby supplying data on the relative positions of the different parts of a robot with respect to the environment. This information is typically published on the global topics `/tf` and `/tf_static`, however, in a multi-robot context, it is necessary to partition this information by redirecting it to local topics prefixed with the namespace to prevent overloading the global topics.

Nevertheless, to make all the robot branches visible to the central `world` frame and to enable communication—and consequently collaboration—among the robots, the use of an external node, the `tf_relay`, is required. This node subscribes to the `tf` and `tf_static` topics of each robot and republishes the information onto the global topics.

The second function, `create_namespaced_bridge_yaml`, loads a base Gazebo-ROS bridge YAML file and rewrites the topic names to include a robot-specific namespace. It also converts the message type from `TwistStamped` to `Twist`, as Gazebo requires the latter format for velocity messages. In this function, it can be observed that the bridge of the Gazebo clock is not implemented because it is not possible to create a different topic that counts the simulation time, in fact the bridge of

the clock is set externally with the `clock_node` in the launch files. Note that each function and node specifies the `'use_sim_time': 'true'` argument in order to synchronize all processes to Gazebo's simulation time.

Subsequently, it can be observed in the spawn file that passing the namespace string enables a robot to be launched in the simulation with its specific name. This is followed by a `GroupAction` function used to apply the same setting to the publisher and SLAM nodes, which require `PushRosNamespace` to inject the namespace into the ROS context of the involved nodes. Additionally, the remapping of the `/tf` and `/tf_static` topics is performed to ensure the correct publication of information into local topics. Finally, the custom start node of the Nav2 navigation stack is launched, which, through the `RewrittenYaml` function, rewrites the temporary parameters (placeholders) of the YAML configuration file with the robot namespace to use a single configuration file for all of them.

Chapter 4

Exploration Logics

The exploration strategies used for mapping follow two approaches: one stochastic and one deterministic. Both rely on a global map provided by the map merging node, which enables the implementation of centralized logic in both cases.

4.1 Map Merger

The map merger (`map_merger.py`) implements a ROS 2 node that merges the local maps of the three robots into a single global map.

The node subscribes to the local maps published by each robot (`/robotX/map`), extracting the coordinates (in meters) and the values of the known cells (free space or obstacles) relative to the global frame "world". In this way, it is possible to determine the current dimensions of the global map based on the collected boundary indexes.

Each local map identifies an obstacle cell with the value of 100, a free cell with 0, and an unknown cell with -1. The underlying logic involves using masks consisting of free and occupied cells to build the global map. Specifically, the obstacle mask is applied first; subsequently, the free-space mask overwrites any cells temporarily identified as occupied due to the presence of moving obstacles.

Once the construction of the global map is complete, it is published as an `OccupancyGrid` message on the `/map` topic at a frequency of 1 Hz.

4.2 Stochastic Exploration

Stochastic exploration is performed using the `stochastic_explorer.py` node, which implements a ROS 2 node that coordinates the exploration of a swarm of three robots.

The first operation performed is subscribing to the topic where the global map is published, which is essential for the exploration logic, as the target goals assigned to the robots are randomly sampled from it.

For the goal to be considered valid, however, it must satisfy specific criteria. Specifically, it must maintain a distance of at least 6 cells from any point identified as an obstacle, while additionally requiring the proximity of free space within a 2-cell radius. An additional check is performed on the distance of the goal relative to the current position of the robots, as a limitation of LiDAR sensors is their minimum scanning range.

Each valid goal is managed by a coordinator, which assigns it to the nearest robot based on its Euclidean distance. If the selected robot is already occupied, the goal is discarded. The coordinator also monitors the number of goal assignment attempts, which, upon exceeding a certain threshold, declares the absence of frontiers.

This algorithm uses several timers to manage different events. In the first timer, the system checks every 4 seconds for a potential robot stall, which is identified if the robot does not move more than 5 cm during this period. Another timer manages the end of the exploration by counting for 30 seconds (6 ticks of 5 seconds) if no further goals are received and all robots are waiting, otherwise, it resets the counter.

Upon completion, the exploration details are printed, and the node is shut down.

4.3 Deterministic Exploration

Deterministic exploration is performed using the `deterministic_explorer.py` node, which implements a ROS 2 node that coordinates the exploration of a swarm of three robots.

The first operation performed is subscribing to the topic where the global map is published, which is essential for the exploration logic, as the target goals assigned to the robots are chosen from a pool of candidate frontier points from it.

The number of candidates to be evaluated is directly proportional to the current size of the frontier. The frontier is divided into several clusters, and the centroid of each is calculated. Finally, for each cluster, the unexplored pixel closest to the centroid that is adjacent to at least one free cell is selected as a candidate.

For the goal to be considered valid, however, it must satisfy specific criteria. Specifically, a coordinator checks the distance of the goal relative to the current position of the robots, as a limitation of LiDAR sensors is their minimum scanning range. Subsequently, the path cost of each candidate goal relative to each idle robot is evaluated using the A* algorithm, and only the candidate with the lowest cost is selected. The cost computation is a cumulative penalty process where traversing occupied cells is avoided and passing through unknown cells is penalized. An additional penalty is applied to paths that involve passing too close to another robot's position. Furthermore, if the selected goal falls within the repulsion radius (set to 1 meter) of another active goal, a penalty is added to its cost, rendering it inadmissible because it becomes too costly.

Finally, the coordinator checks the best obtained frontiers and verifies that they do not exceed a defined cost threshold, if they do, they are discarded as inaccessible. Otherwise, the goal is assigned to the available robot.

The algorithm uses the same timers as the previous algorithm to manage stalls and the termination of the exploration.

Chapter 5

Simulations

The final phase of the project involves simulating the complete operation of the exploration algorithms within Gazebo Harmonic. Using RViz, it was possible to monitor goal assignments, robot paths, and the progress of environmental mapping in real time.

This phase was crucial for identifying critical issues in the project, which led to several modifications compared to the initial versions.

First, an issue regarding a delay in the TF tree construction was occasionally encountered. This resulted in a startup failure of the exploration node due to a missing connection between the global frame "world" and the robot frame "robotX/base_footprint". To resolve this issue, an initial startup check was implemented within the exploration nodes, allowing the coordinator to start only when the TF tree is completely connected.

Another problem detected during the simulations concerned the `cost_scaling_factor` of obstacles in the global costmap. The global costmap is responsible for planning the path while avoiding obstacles; however, with a reduced `cost_scaling_factor`, as can be seen in Figure 5.2, obstacles occupy a large portion of the traversable space, compromising the generation of safe paths. To address this issue, this parameter was increased in the Nav2 configuration file.

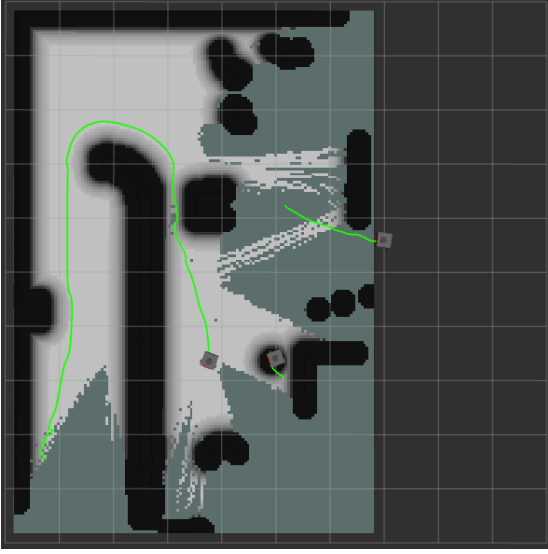


Figure 5.1: Modified Global Costmap of robot3
(cost_scaling_factor = 8.0)

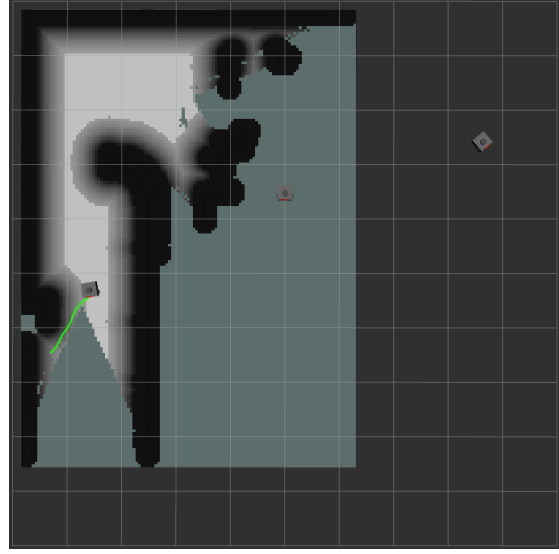


Figure 5.2: Default Global Costmap of robot3
(cost_scaling_factor = 3.0)

During certain simulations, a substantial computational bottleneck caused by CPU overload was observed, as the management of multiple robots, the rewriting of the global map at each cycle, and the entire simulated environment introduce unavoidable process synchronization issues. Indeed, on several occasions, the robots collided with obstacles (both static and dynamic), a situation that should intrinsically be prevented by the use of SLAM and Nav2 through the constant updating of their local costmaps.

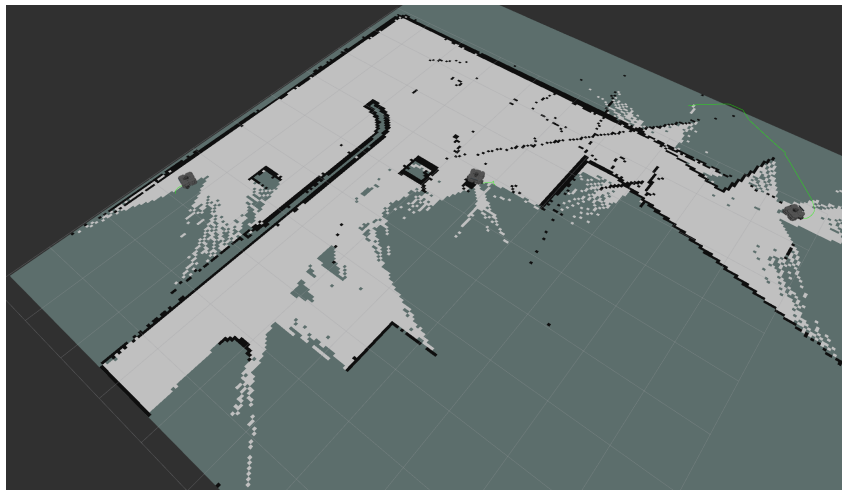


Figure 5.3: Cursed Global Map

In these cases, the SLAM instances of the robots involved in the collision could become permanently compromised, subsequently corrupting the global map. This behavior occurs even more frequently when employing deterministic exploration, as the CPU is further strained by the computational demands of the A* algorithm. Lastly, a final refinement to the merged map involves eliminating temporary obstacles caused by the mutual detection of the robots. To achieve this, an overwriting priority order was established for the global map, dictating that obstacles are written onto unknown space first, and only subsequently are free spaces written. In this manner, if a robot explores an area where a moving obstacle was previously detected, the latter will no longer be observed, and the obstacle will be successfully cleared from the global map.

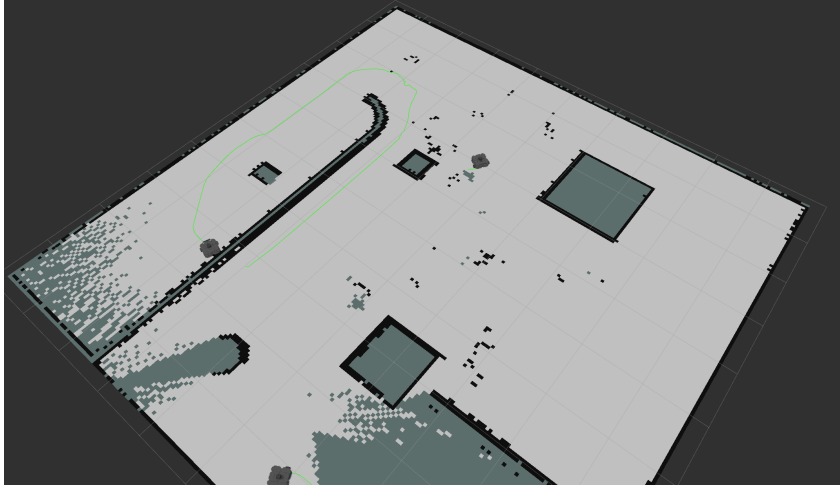


Figure 5.4: Dirty Global Map

Conclusions

This project successfully implemented and validated a multi-robot autonomous mapping system in an unknown environment, evaluating two distinct exploration strategies. To compare the two approaches, performance metrics such as exploration duration, individual robot distance, and the total distance covered by the swarm were analyzed.

While stochastic exploration is inherently random and yields highly variable metrics across different simulations, one would expect the deterministic approach to consistently produce identical results in terms of metrics, paths, and assigned goals. However, this is not the case due to non-deterministic factors influencing the process, such as inter-node communication latencies, sensor noise, and thread scheduling variations caused by CPU load. Despite these variations, it was still possible to conduct a meaningful comparative analysis between the two strategies.

In particular, it can be noted that exploration takes less time with stochastic logic than with deterministic logic. On the other hand, robots tend to travel greater distances in the first scenario compared to the second. This is attributable to the underlying logics themselves: in stochastic logic, the candidate goal is randomly selected and might be quite distant, even from the closest robot. Consequently, the robots must cover large distances relative to their starting positions to reach their goals. In the case of deterministic exploration, the goal is almost always in the robot's immediate vicinity; however, upon reaching it, the robot remains idle while waiting for a new assignment, resulting in a longer overall exploration time.

Conclusions

```
[swarm_ex-50] [INFO] [1783243898.790089799] [swarm_explorer]: No frontiers available. Shutting down... (6/6)
[swarm_ex-50] [INFO] [1783243898.790665872] [swarm_explorer]: =====
[swarm_ex-50] [INFO] [1783243898.791068539] [swarm_explorer]: Total duration: 2 min and 58 sec (178.00 s)
[swarm_ex-50] [INFO] [1783243898.791440332] [swarm_explorer]: Distance traveled:
[swarm_ex-50] [INFO] [1783243898.791865056] [swarm_explorer]:   - robot1: 23.70 meters
[swarm_ex-50] [INFO] [1783243898.792357453] [swarm_explorer]:   - robot2: 17.45 meters
[swarm_ex-50] [INFO] [1783243898.792736544] [swarm_explorer]:   - robot3: 20.12 meters
[swarm_ex-50] [INFO] [1783243898.793161119] [swarm_explorer]: Total swarm distance: 61.27 meters
[swarm_ex-50] [INFO] [1783243898.793596056] [swarm_explorer]: =====
[swarm_ex-50] [INFO] [1783243898.794027859] [swarm_explorer]: Exploration Complete. All accessible areas have been mapped.
```

Figure 5.5: Stochastic Explorer Metrics

```
[swarm_ex2-50] [INFO] [1783328927.499890225] [swarm_explorer2]: No frontiers available. Shutting down... (6/6)
[swarm_ex2-50] [INFO] [1783328927.502038133] [swarm_explorer2]: =====
[swarm_ex2-50] [INFO] [1783328927.503542576] [swarm_explorer2]: Total duration: 6 min and 51 sec (411.00 s)
[swarm_ex2-50] [INFO] [1783328927.504802886] [swarm_explorer2]: Distance traveled:
[swarm_ex2-50] [INFO] [1783328927.506004405] [swarm_explorer2]:   - robot1: 17.23 meters
[swarm_ex2-50] [INFO] [1783328927.506640984] [swarm_explorer2]:   - robot2: 12.40 meters
[swarm_ex2-50] [INFO] [1783328927.508500904] [swarm_explorer2]:   - robot3: 14.11 meters
[swarm_ex2-50] [INFO] [1783328927.509106702] [swarm_explorer2]: Total swarm distance: 43.74 meters
[swarm_ex2-50] [INFO] [1783328927.509831937] [swarm_explorer2]: =====
[swarm_ex2-50] [INFO] [1783328927.511414020] [swarm_explorer2]: Exploration Complete. All accessible areas have been mapped.
```

Figure 5.6: Deterministic Explorer Metrics

The maps obtained from the exploration using the two algorithms are presented below.

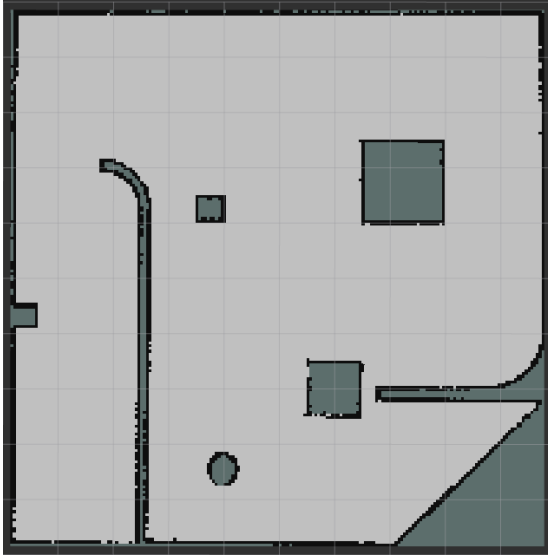


Figure 5.7: Stochastic Exploration map

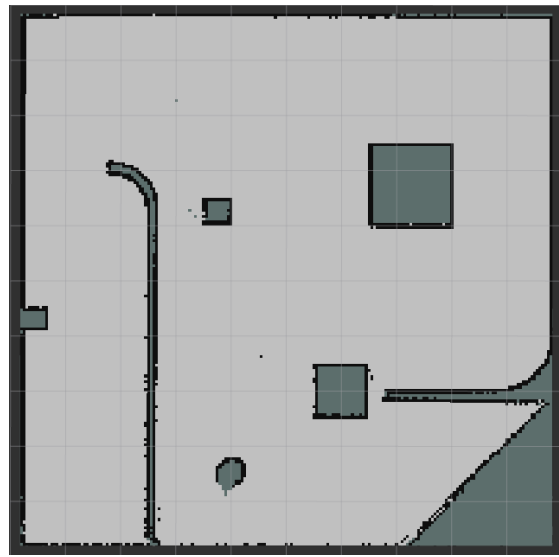


Figure 5.8: Deterministic Exploration map

Despite the good results achieved by the project, there remains room for improvement. Firstly, the map merger's computational efficiency could be improved by replacing the current approach with an incremental one, updating the global map

at each iteration rather than rewriting it completely.

As for the stochastic exploration algorithm, a highly uncommon yet possible issue is the assignment of a goal outside the robot’s local map. This requires addressing the edge case by implementing, for instance, a progressive logic; this approach would keep the ultimate goal in mind while assigning intermediate waypoints close enough to the robot to steer it toward the target.

Lastly, the deterministic algorithm could also be optimized by decreasing the number of candidate centroids for goal selection. This would simultaneously increase computational efficiency and further mitigate the risk of assigning overlapping or nearby goals to multiple robots.